

Tests

Mastermind verification report

The submitted source compiled successfully, and a deterministic Java harness completed all 12 tests. The suite checks the feedback algorithm, validation, command handling, attempt accounting, win and loss outcomes, testing mode, and a nondefault configuration.

Requirement coverage

Assignment requirement	Test IDs	Evidence	Result
Duplicate-color feedback	T01-T05	Exact matches take precedence; colors and pegs are never counted twice.	PASS
Invalid input	T07-T09	Wrong length and illegal symbols are rejected; attempts remain unchanged.	PASS
History	T09	Only valid guesses appear, in order; HISTORY does not consume an attempt.	PASS
Win and loss	T09-T11	Win count, attempt limit, and secret disclosure are correct.	PASS
Testing mode	T11	Fixed secret and REVEAL are deterministic and do not alter attempt accounting.	PASS
Changed configuration	T12	5-symbol code, 3-symbol alphabet, and 6-attempt limit work without engine changes.	PASS

Method: tests T01-T08 call the evaluator or validator directly. Tests T09-T12 run GuessingGameEngine with a fixed secret and scripted UserInterface. A separate console run launched assignment2.Driver mastermind test to confirm the real prompts and command path.

Feedback algorithm tests

These cases target the rule most likely to fail: exact matches must be removed before color-only matches, and each peg can contribute at most once.

ID	Scenario	Secret	Guess	Expected	Observed	Purpose
T01	All colors reordered	BGOR	RBGO	0B_4W	0B_4W	Counts every color once when no position matches.
T02	Guess repeats a color too often	AABB	AAAA	2B_0W	2B_0W	Prevents extra A values from becoming white pegs.
T03	Duplicate colors swapped	AABB	BBAA	0B_4W	0B_4W	Finds all valid white pegs with duplicates.
T04	Secret contains two copies	RRGY	RRRR	2B_0W	2B_0W	Consumes both exact R matches before the white pass.
T05	Exact-match precedence	GRRY	RRRR	2B_0W	2B_0W	Confirms exact positions are resolved first.
T06	No shared colors	BGOR	PYPY	0B_0W	0B_0W	Provides a zero-overlap baseline.

Evaluator rule confirmed

MastermindEvaluator first marks black pegs by position. It then builds counts only from unmatched secret symbols and decrements a count whenever an unmatched guess earns a white peg. This sequence explains the observed results and prevents double counting.

Validation and game-flow tests

ID	Focus	Setup and input	Expected and observed	Result
T07	Wrong length	Default config; input BG	Rejected with: Guess must contain exactly 4 symbols.	PASS
T08	Illegal symbol	Default config; input BGOZ	Rejected because Z is not in [B, G, O, P, R, Y].	PASS
T09	Invalid input and HISTORY	Secret PYYY; inputs ZZZZ, BG, BGOR, HISTORY, PYYY	Invalid inputs stay on attempt 1. HISTORY lists 1. BGOR -> 0B_0W. The next prompt remains attempt 2, then the player wins in 2 valid guesses.	PASS
T10	Loss at attempt limit	Secret BGOR; twelve valid guesses of YYYY	Prompt reaches attempt 12/12, then reports loss and reveals BGOR.	PASS
T11	Testing mode and win	Secret BGOR; inputs REVEAL, BBBB, BGOR	REVEAL repeats the secret without consuming an attempt. Feedback is 1B_0W then 4B_0W; win reports 2/12 attempts.	PASS

Representative console evidence

Step	Observed output
Invalid symbol	Attempt 1/12 ... Invalid guess: 'Z' is not a legal symbol.
Wrong length	Attempt 1/12 ... Invalid guess: Guess must contain exactly 4 symbols.
History command	Attempt 2/12 ... 1. YYYY -> 0B_0W ... then Attempt 2/12 again
Deterministic win	Feedback: 4B_0W ... You won! The code was BGOR. Attempts used: 2/12.

Changed-configuration test

T12 verifies that all three configuration dimensions can change without modifying GuessingGameEngine, DefaultGuessValidator, or either secret generator.

Property	Default	T12 configuration
Maximum attempts	12	6
Code length	4	5
Legal symbols	B, G, O, P, R, Y	1, 2, 3
Fixed secret	not applicable	11111

Attempt	Guess	Expected	Observed	Outcome
1	22222	0B_0W	0B_0W	Continue
2	11122	3B_0W	3B_0W	Continue
3	11111	5B_0W	5B_0W	Win at 3/6

Result

All required behavior is covered by at least one passing test, and the highest-risk feedback logic is covered by five duplicate-color cases. The changed configuration demonstrates that attempt count, code length, and alphabet are configuration data rather than hard-coded engine behavior.

Automated harness	Console path	Submission status
12/12 passed	Driver scenario passed	Verified against the submitted source

Tests and results

Part II verification | 20 deterministic automated checks passed; source compiled with OpenJDK 17.

Run from the project root: `javac -d out $(find src test -name "*.java")` then `java -cp out assignment2.Phase2Tests`

Wordle scoring / per-position marks

ID	Secret	Guess	Expected = observed	Purpose
W01	APPLE	APPLE	C C C C C	Win when every position matches.
W02	APPLE	ALLEY	C P A P A	Excess L is absent; E is present.
W03	APPLE	ALARM	C P A A A	Correct A consumes its only copy.
W04	APPLE	PAPAL	P P C A P	Exact P takes priority over present P.
W05	BANAL	LLAMA	P A P A P	Two As and one L cannot be overused.
W06	APPLE	GHOST	A A A A A	No shared letters.

Validation and loading

ID	Scenario	Expected = observed
W07	A1PLE; APP	Nonletters and wrong length rejected.
W08	ZZZZZ; lowercase apple	Unlisted word rejected; case normalized to APPLE.
W09	Seeded random generator	Selected secret occurs in the dictionary.
W10	Empty or malformed word list	Startup rejects invalid dictionary content.

Wordle game flow

A single round with an APPLE secret

ID	Input sequence / setup	Assertion
W11	AA, A1PLE, ZZZZZ, ALLEY, HISTORY, REVEAL, apple	Attempt 1 repeats after all three invalid inputs. HISTORY and REVEAL leave the attempt at 2. Final valid guess wins.
W12	Same scripted round	HISTORY shows only "1. ALLEY -> C P A P A"; REVEAL prints APPLE.
W13	Six GHOST guesses against APPLE	Exactly six valid guesses cause a loss and reveal APPLE.
W14	Test mode: APPLE win, Y, BANAL win, N	A second round uses a new fixed secret and resets to attempt 1.

Why APPLE / ALLEY differs from the handout

The handout prints C P P A A, but that contradicts its own one-occurrence and positional rules. APPLE contains one L, so only the first unmatched L in ALLEY can earn P. E exists in APPLE at a different position and must earn P. The rule-consistent result is C P A P A. W02 asserts the written Wordle rule.

Console path

The real Driver was also launched for both game names in test mode with fixed secrets. It loads data/words.txt from disk for Wordle, prints positional feedback, and accepts replay. The automated harness calls the same engine and session classes through a scripted UserInterface.

Mastermind regression

The Phase I game remains runnable from the same Driver and preserves its black/white peg format.

ID	Case	Expected = observed
M01	AABB vs AAAA	2B_0W: extra As do not count.
M02	BGOR vs RBGO	0B_4W: all pegs white.
M03	BG; BGOZ under default rules	Length and illegal color rejected.
M04	Invalid ZZZZ, valid BGOR, HISTORY, then PYYY	History lists only BGOR. Invalid guess and HISTORY do not cost attempts; PYYY wins.
M05	Fixed BGOR win, replay, fixed PYYY win	Two independent rounds and two wins.
M06	Twelve YYYY guesses vs BGOR	Loss occurs at 12/12 and reveals BGOR.

Coverage map

Requirement	Tests
Wordle duplicate accounting / exact priority	W02-W05
Dictionary, length, letters, random secret	W07-W10
Invalid attempts, HISTORY, REVEAL	W11-W12
Win, loss, replay, deterministic test mode	W01, W11, W13-W14
Mastermind feedback, input, commands, outcomes	M01-M06

Result: 20 / 20 passed with OpenJDK 17 on September 23, 2026. Run the harness after changing the dictionary or source code. The source harness is included under test/assignment2/Phase2Tests.java.